

EC-333 Microprocessor and Interfacing Techniques

Experiment # 2

Program Structure and Addressing Modes.

1. Objective/s:

- Introduction to Assembly Language
- Memory Models
- Program Structures
- Addressing Modes
- Example Codes
- Lab Task

2. Memory Models

Specifies the memory configuration for the assembly language program.

2.1. TINY Model:

In the TINY model, both code and data occupy one physical segment of 64KByte. Therefore, all procedures and variables are by default addressed as NEAR, by pointing at their offsets in the segment.

2.2. SMALL Model:

In the SMALL model, all of the code is placed in one physical segment of 64KB and all data in another physical segment of 64KB. In this model, all procedures and variables are addressed as NEAR by pointing to their offsets only.

2.3. COMPACT Model:

In the COMPACT model, all elements of code (e.g. procedures) are placed into one physical segment. However, each element of data can be placed by default into its own physical segment. Consequently, data elements are addressed by pointing both at the segment and offset addresses. In this model, all code elements (procedures) are addressed as NEAR and data elements (variables) are addressed as FAR.

2.4. MEDIUM Model:

The MEDIUM model is the opposite of the compact model. In this model data elements are treated as NEAR and code elements are addressed as FAR.

2.5. LARGE Model:

In the LARGE model both code elements (procedures) and data elements (variables) are put in different physical segments. Procedures and variables are addressed as FAR by pointing at both the segment and offset addresses that contain those elements. However, no data array can have a size that exceeds one physical segment (i.e. 64 KB).

2.6. HUGE Model:

The HUGE memory is similar to the LARGE model with the exception that a data array may have a size that exceeds one physical segment (i.e. 64 KB).

The following table summarizes the use of models and the number and sizes of physical segments that are used with each of the models.

Memory Model	Size of Code	Size of Data
TINY	Code + Data < 64KB	Code + data < 64KB
SMALL	Less than 64KB	Less than 64KB
MEDIUM	Can be more than 64KB	Less than 64 KB
COMPACT	Less than 64KB	Can be more than 64KB
LARGE*	Can be more than 64K	Can be more than 64KB
HUGE**	Can be more than 64K	Can be more than 64KB

Table 1: Memory Models

3. Program Structure

Assembly language programs divide roughly into five sections

- Header
- Equates
- Data (Data Segment)
- Body (Code Segment)

3.1. Header

The header contains various directives, which do not produce machine code

Some of the sample headers are:

```
%TITLE "Sample Header"  
.8086  
.model small  
.stack 256
```

3.2. Equates

Constant values are known as *equate*, they are just like constants used in the c/c++ these days. Key word EQU is used to define an equate. Some of the sample equates are:

```
Count EQU 10  
Element EQU 5  
Size = Count * Element  
MyString EQU "Maze of twisty passages"  
Size = 0
```

Note that,

- = is used for numeric values only
- Cannot change value of EQU symbol
- EQUated symbols are not variables
- EQU expressions are evaluated where used; = expressions are evaluated where defined

3.3. Data Segment

It is the segment where all the variables are declared/defined/initialized. Data Segment begins with the *.data* directive. There are two kinds of variables, *initialized* and *uninitialized*. Initialized variables take up space in the program's code file while uninitialized variables do not take up space in the program's code file. A Sample DATA SEGMENT is shown below.

```
.data  
numRows DB 25  
numColumns DB ?  
videoBase DW 0800h
```

- DB and DW are common directives (define byte) and (define word)
- The symbols associated with variables are called *labels*
- Strings may be declared using the DB directive:

```
aTOm DB "ABCDEFGHIJKLM"
```

- Pseudo-ops to define data or reserve storage

- DB - byte(s)
 - DW - word(s)
 - DD - doubleword(s)
 - DQ - quadword(s)
 - DT - tenbyte(s)
- Numeric data values
 - 100 - decimal
 - 100b - binary
 - 100h - hexadecimal
 - '100' - ASCII
 - "100" - ASCII

3.3.1. Arrays

Any consecutive storage locations of the same size can be called as an array

```
X  DW  040Ch, 10b, -13, 0
Y  DB  'This is an array'
Z  DD  -109236, FFFFFFFFh, -1, 100b
```

- Components of X are at X, X+2, X+4, X+6
- Components of Y are at Y, Y+8, Y+16, Y+24
- Components of Z are at Z, Z+4, Z+8, Z+12

3.4. Programming Body

This is the block where we write our code, also known as the *code segment*. It is divided into four columns: labels, mnemonics, operands, and comments. Following is an example.

Label	Mnemonic	Operand	Comment
	.data		
exCode	DB	0	;A byte variable
myWord	DW	?	;Uninitialized word var.
	.code		
MAIN	PROC		
	mov	ax, @data	;Initialize DS to address
	mov	ds, ax	; of data segment
	jmp	Exit	;Jump to Exit label
	mov	cx, 10	;This line skipped!
Exit:	mov	ah, 04Ch	;DOS function: Exit prog
	mov	al, exCode	;Return exit code value

```

                int          21h          ;Call DOS. Terminate prog
MAIN           ENDP          ;End Program
                END          MAIN         ; and specify entry point

```

3.4.1. The Label Field

- Labels mark places in a program which other instructions and directives reference
- Labels in the code segment always end with a colon
- Labels in the data segment never end with a colon
- Labels can be from 1 to 31 characters long and may consist of letters, digits, and the special characters `? . @ _ $ %`
- If a period is used, it must be the first character
- Labels must not begin with a digit
- The assembler is *case insensitive*

3.4.2. The Mnemonic Field

- For an instruction, the operation field contains a symbolic operation code (*opcode*)
- The assembler translates a symbolic opcode into a machine language opcode
- Examples are: ADD, MOV, SUB
- In an assembler directive, the operation field contains a directive (*pseudo-op*)
- Pseudo-ops are not translated into machine code; they tell the assembler to do something

3.4.3. The Operand Field

- For an instruction, the operand field specifies the data that are to be acted on by the instruction. May have zero, one, or two operands

```

NOP                ;no operands -- does nothing
INC AX             ;one operand -- adds 1 to the contents of
AX
ADD WORD1,2        ;two operands -- adds 2 to the contents
                   ; of memory word WORD1

```

- In a two-operand instruction, the first operand is the *destination operand*. The second operand is the *source operand*.
- For an assembler directive, the operand field usually contains more information about the directive.

3.4.4. The Comment Field

- A semicolon marks the beginning of a comment field
- The assembler ignores anything typed after the semicolon on that line

- It is almost impossible to understand an assembly language program without good comments
- Good programming practice dictates a comment on almost every line

4. Addressing Modes

The different ways in which a source operand is denoted in an instruction is known as *addressing modes*. There are 8 different addressing modes in 8086 programming, which are explained one by one:

4.1. Immediate Addressing Mode

The addressing mode in which the data operand is a part of the instruction itself is known as immediate addressing mode.

Example

```
MOV CX, 4929H
ADD AX, 2387H
MOV AL, FFH
```

4.2. Register Addressing Mode

It means that the register is the source of an operand for an instruction.

Example

```
MOV CX, AX ; copies the contents of the 16-bit AX register into the 16-bit CX register),
ADD BX, AX
```

4.3. Direct Addressing Mode

The addressing mode in which the effective address of the memory location is written directly in the instruction.

Example

```
MOV AX, [1592H]
MOV AL, [0300H]
```

4.4. Register Indirect Addressing Mode

This addressing mode allows data to be addressed at any memory location through an offset address held in any of the following registers: BP, BX, DI & SI.

Example

```
MOV AX, [BX] ; Suppose the register BX contains 4895H, then the contents 4895H are mvd to AX
ADD CX, [BX]
```

4.5. Based Addressing Mode

In this addressing mode, the offset address of the operand is given by the sum of contents of the BX registers and 8-bit displacement.

Example

```
MOV DX, [BX+04]
ADD CL, [BX+08]
```

4.6. Indexed Addressing Mode

In this addressing mode, the operands offset address is found by adding the contents of SI or DI register and 8-bit/16-bit displacements.

Example

```
MOV BX, [SI+16]
ADD AL, [DI+16]
```

4.7. Based-Index Addressing Mode

In this addressing mode, the offset address of the operand is computed by summing the base register to the contents of an Index register.

Example

```
ADD CX, [AX+SI]
MOV AX, [AX+DI]
```

4.8. Based Indexed With Displacement Mode

In this addressing mode, the operands offset is computed by adding the base register contents. An Index registers contents and 8 or 16-bit displacement.

Example

```
MOV AX, [BX+DI+08]
ADD CX, [BX+SI+16]
```

5. Tasks

5.1. Direct Addressing and Arithmetic Operations. Write codes for the following

1. Convert your year of birth to a hexadecimal number and store write it to memory address 0200h. Is 1 byte enough to store the number? Why/why not? Do the writing to memory accordingly.

2. Get the last 4 digits of your student number and write it to next memory address.
Consider which memory address you should be using according to the answer of the first question.
3. Find the sum of these two numbers. Store the result in register DX.
4. Subtract first number from second store the result in register CX.

5.2. Indirect Addressing and Arithmetic Operations

1. Store 50 (decimal) in memory address 0300h, this time use indirect addressing (access with [BX]). Is 1 byte enough to store the number? Why/why not? Do the writing to memory accordingly.
2. Store 51 in next address. Consider which memory address you should be using according to the answer of the first question.
3. Store 52 in next address.
4. Find the sum of 3 numbers. Store the result in the next address.

5.3. Advanced Data Transfer

Analyze the assembly code below:

```
title Register test program

.model small
.stack 100h
.data
msg db "AB",0dh,0ah,'$'

.code
main proc
    mov ax,@data
    mov ds,ax
; Add your code here
; -----
; -----
    mov ah,9
    mov dx, offset msg
    int 21h
```

The program above prints “AB” to the console. The message “AB” is stored at msg in data segment. Therefore, “offset msg” is the starting address of the string. Modify the code according to the following experiments (**Try both MOV and LEA commands**):

- 1- Copy the second byte stored at “msg” (**which is ‘B’ currently**) to the first byte. The output should be “BB”. In order to do that, load 2-bytes data stored at “msg” into a **16-bits** register. Copy one part of the register to the other part. Then, write your register back to msg.
- 2- Copy the first byte stored at “msg” (**which is ‘A’ currently**) to the second byte. The output should be “AA”. To achieve this, load the first byte stored at “msg” into one **8-bits** register (high or low part) and copy it onto the other part and write the data stored in register back to msg.
- 3- Load the data stored at “msg” into one register and swap the high and low parts of the register using a temporary **8-bits** register. Then write back your “swapped” register onto msg. The output should be “BA”.

- **Task 2 is evaluated in your lab report.**